Gradient Inversion Attacks in Federated Learning

Deep Leakage from Gradients: Reconstructing Client Data from Shared Updates

Contents

1 Introduction **2**

2 Related Work **3**
2.1 Federated Learning . . . 3
2.2 Optimization-Based Gradient Inversion . . . 3
2.3 Closed-Form Gradient Inversion . . . 3
2.4 Defenses Against Gradient Inversion . . . 4
2.5 Positioning of This Project . . . 4

3 Problem Statement **5**
3.1 Federated Learning Setting . . . 5
3.2 Threat Model . . . 5
3.3 The Gradient Inversion Problem . . . 6

4 Methodology **7**
4.1 Victim Architectures . . . 7
4.2 DLG and iDLG . . . 7
4.3 Inverting Gradients . . . 8
4.4 R-GAP . . . 8
4.5 DP-SGD as a Defense . . . 9

5 Experimental Evaluation **10**
5.1 Evaluation Objectives . . . 10
5.2 Evaluation Scenarios . . . 10
5.2.1 Datasets and Victim Configurations . . . 10
5.2.2 Evaluation Metrics . . . 10
5.3 Results and Discussion . . . 10
5.3.1 RQ1 – Baseline Comparison on a Shared Architecture . . . 10
5.3.2 RQ2 – Scaling to a Deeper Architecture . . . 10
5.3.3 RQ3 – Robustness to Batch Size . . . 10
5.3.4 RQ4 – Effectiveness of the DP-SGD Defense . . . 10

Chapter 1

Introduction

Federated Learning (FL) is a distributed training setup where many clients (e.g. phones, hospitals, organizations) collaboratively train one shared model without pooling their data on a central server - each client trains locally on its own private data and only sends back the resulting model update [1]. This is often presented as a privacy-preserving alternative to centralized training: since raw data never leaves the client, FL is commonly assumed to keep that data confidential by construction.[1]

This project studies whether that assumption actually holds. The gradient (or weight update) a client sends is not an opaque summary - it is the deterministic output of backpropagating a loss on the client's private batch through a model architecture the server already knows. If that function can be inverted, the private batch can be reconstructed from the gradient alone, without any further access to the client. This is the *gradient inversion* problem, first demonstrated as "Deep Leakage from Gradients" [2]: given a leaked gradient and a known model architecture, recover the input/label pair that produced it.

This project focuses on two broad attack strategies: *optimization-based* attacks, which search for an input/label pair that reproduces the leaked gradient, and *closed-form* attacks, which solve for it directly from the per-layer gradient equations, with no search involved. It implements one representative method from each - DLG [2]/iDLG [3] and Inverting Gradients [4] on the optimization side, R-GAP [5] on the closed-form side - on a common set of victim architectures, and evaluates both families against a DP-SGD-style [6] gradient perturbation defense.

The remainder of this report is organized as follows. Chapter 2 surveys the work this project builds on. Chapter 3 gives a formal, input/output statement of the FL setting, the threat model, and the gradient inversion problem. Chapter 4 describes the attack and defense methods implemented. Chapter 5 presents the experimental evaluation methodology and results, and Chapter 6 concludes with lessons learned.

[1] Code, experiment notebooks, and the LaTeX source of this report are available at `https://github.com/huynt119/Trustworthy_AI_Project`.

Chapter 2

Related Work

2.1 Federated Learning

FedAvg [1] is the foundational FL algorithm and the training protocol assumed throughout this project: clients run several steps of local SGD on their own data and the server aggregates the resulting updates by a weighted average (Equation 3.2 in Chapter 3). Its privacy argument is informal - clients never transmit raw data, only model updates - and is exactly the assumption the gradient inversion literature below sets out to test.

2.2 Optimization-Based Gradient Inversion

Deep Leakage from Gradients (DLG) [2] was the first to show that a single gradient can leak the private image and label that produced it. DLG treats reconstruction as an optimization problem: a dummy input and label are updated by gradient descent so that their gradient matches the leaked one under an L_2 distance, using second-order differentiation ("double backpropagation") to backpropagate through the gradient-matching loss itself. Joint optimization of the dummy label alongside the image makes convergence slow and occasionally unstable.

iDLG [3] removes that instability by observing that, for cross-entropy loss on a one-hot label, the sign pattern of the last layer's gradient identifies the ground-truth class in closed form - only the dummy image still needs to be optimized. This simplification makes convergence markedly more reliable.

Inverting Gradients [4] argues that most of the information in a gradient is carried by its *direction* rather than its magnitude, and replaces the L_2 distance with a cosine-similarity cost. It further adds a total-variation image prior and optimizes with a sign-based variant of Adam. These changes make the attack scale from small networks up to ImageNet-scale ResNet architectures, showing that gradient leakage is a practical threat even for deep convolutional networks trained with non-trivial batch sizes.

2.3 Closed-Form Gradient Inversion

R-GAP (Recursive Gradient Attack on Privacy) [5] reformulates gradient leakage as a linear algebra problem rather than a search problem. For a feed-forward network with bias-free layers and invertible activations, the weight-gradient equation at each layer relates a known quantity (the leaked per-layer weight gradient) to the layer's input, which can be solved for directly; recursing this from the output layer back to the input recovers the private sample with no optimization

loop. R-GAP additionally proposes a *rank analysis* that predicts, from the architecture alone, whether a given layer's input is uniquely recoverable, without running the attack at all.

2.4 Defenses Against Gradient Inversion

DP-SGD [6] was originally proposed to give differential-privacy guarantees during training, by clipping each per-example gradient to a norm C and adding Gaussian noise with multiplier σ. The same clip-and-noise procedure is widely reused, as in this project, as an ad hoc empirical defense against gradient inversion: since it is applied directly to the gradient the attacker observes, it perturbs the attacker's input rather than any property of the model.

2.5 Positioning of This Project

This project contributes an empirical comparison, under matched conditions, of optimization-based attacks (DLG/iDLG, and separately Inverting Gradients) and a closed-form attack (R-GAP) across comparable victim architectures, together with an evaluation of how all three degrade under the same DP-SGD defense budget.

Chapter 3

Problem Statement

3.1 Federated Learning Setting

FL trains a shared model across K clients, where client k keeps a private dataset $\mathcal{D}_k$ and the server optimizes the global objective

$$\min_{w} \; F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w), \qquad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(w; x_i, y_i), \tag{3.1}$$

where w are the shared parameters and $n_k = |\mathcal{D}_k|$, $n = \sum_k n_k$. In round t, the server sends w_t to a subset of clients; each client trains locally and returns an update $\Delta w_k^{(t)}$ (or, in the single-step case this project focuses on, a gradient $g_k^{(t)} = \nabla_w F_k(w_t)$ scaled by the learning rate). The server aggregates with the FedAvg rule

$$w_{t+1} = w_t + \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \Delta w_k^{(t)}. \tag{3.2}$$

Only $\Delta w_k^{(t)}$ (equivalently $g_k^{(t)}$) ever leaves the client - never x_i or y_i directly.

3.2 Threat Model

We consider an *honest-but-curious* attacker: it follows the FL protocol exactly as specified, but additionally tries to extract private data from whatever it legitimately observes. Two placements are relevant, and both reduce to the same observation:

- **A curious server**, which by design receives every client's update $\Delta w_k^{(t)}$ (or gradient $g_k^{(t)}$) each round.
- **An eavesdropper** on the client–server channel, which intercepts the same quantity in transit.

What the attacker knows: the model architecture and loss function $\ell(\cdot\,;\cdot)$, the current global weights w_t, and the leaked gradient (or update) itself. **What the attacker does not know:** the private batch $\{(x_i, y_i)\}$ that produced that gradient. The attacker makes no other assumption and does not deviate from the protocol - it is purely a passive observer of data every FL deployment already produces.

3.3 The Gradient Inversion Problem

Formally, for a private batch of size B,

$$g = \nabla_w \ell\big(w_t;\, \{x_i, y_i\}_{i=1}^{B}\big) \tag{3.3}$$

is the quantity the attacker observes (for $B = 1$ this is simply $\nabla_w \ell(w_t; x, y)$). Equation (3.3) is a deterministic function of the private batch and the known weights w_t; it is not necessarily one-way. The **gradient inversion problem** is to invert it:

> **Input:** the model architecture and loss ℓ, the weights w_t, and the leaked gradient g.
> **Output:** an estimate $\{(\hat{x}_i, \hat{y}_i)\}_{i=1}^{B}$ of the private batch.

The two attack families surveyed in Chapter 2 answer this input/output contract differently. Optimization-based attacks pose it as a search problem,

$$\min_{\hat{x},\hat{y}} \; D\big(g', g\big), \qquad g' = \nabla_w \ell(w_t; \hat{x}, \hat{y}), \tag{3.4}$$

for some distance or similarity measure $D(\cdot, \cdot)$ between gradient vectors, solved by gradient descent on $(\hat{x}, \hat{y})$. Closed-form attacks instead treat Equation (3.3) as a system of per-layer equations and solve directly for $\{\hat{x}_i\}$ by linear algebra, with no search and no loss D to minimize. Chapter 4 details both routes for each method implemented in this project. How the quality of $(\hat{x}, \hat{y})$ is measured against the true (x, y) - and under what gradient-perturbation defense budget - is defined separately in Chapter 5, since it is a property of the evaluation, not of the problem statement itself.

Chapter 4

Methodology

4.1 Victim Architectures

Three victim architectures are used across the attacks studied in this project:

- **LeNet-style CNN** - four convolutional layers followed by a fully-connected head. Two variants are used: the original DLG formulation (bias, sigmoid activations), used for DLG/iDLG and, under the same architecture, initial weights, and image, for Inverting Gradients, providing a common architecture for the three-attack comparison in Section 4.5; and a bias-free, LeakyReLU variant required by R-GAP (referred to here as LeNetRGAP), since R-GAP's per-layer equations (Section 4.4) assume an affine map with no additive bias and an invertible activation; its output layer is a single logit trained with binary logistic loss, since R-GAP's closed-form output-layer inversion is only defined for a scalar, sigmoid-activated loss.
- **ResNet-18** - used for Inverting Gradients, to test attack scalability on a deeper, residual architecture closer to what is deployed in practice.
- **FCN3** - a three-layer, bias-free, fully-connected network (1000, 100, 1 units, LeakyReLU throughout), used alongside LeNetRGAP for R-GAP as a point of comparison: it has no convolutional layers, and therefore no extra forward-pass constraints beyond the per-layer gradient equations themselves (Section 4.4).

4.2 DLG and iDLG

DLG [2] initializes a dummy sample $(\hat{x}, \hat{y})$ from random noise and optimizes

$$\min_{\hat{x},\hat{y}} \ \left\|g' - g\right\|_2^2, \qquad g' = \nabla_w \, \ell(w_t; \hat{x}, \hat{y}), \tag{4.1}$$

by gradient descent on $(\hat{x}, \hat{y})$ jointly. Because g' is itself a gradient, minimizing Equation (4.1) requires differentiating through $\nabla_w \ell$ a second time ("double backpropagation"); both $\hat{x}$ and $\hat{y}$ are free, continuous optimization variables, including the label, which DLG only relaxes to a one-hot vector after optimization.

iDLG [3] removes the label from the optimization. For cross-entropy loss on a one-hot label y, the gradient of the loss with respect to the pre-softmax logit z_c of class c is $\partial \ell / \partial z_c = p_c - \mathbb{1}[c = y]$, where p_c is the predicted probability. Since $p_c \in (0, 1)$, this quantity is negative only for the true class, so

$$\hat{y} = \arg\min_c \ \left(\frac{\partial \ell}{\partial z_c} \right), \tag{4.2}$$

recovers the ground-truth label in closed form from the sign pattern of the last layer's gradient alone. Only $\hat{x}$ is then optimized against Equation (4.1), which converges markedly faster and more reliably than DLG's joint optimization.

4.3 Inverting Gradients

Inverting Gradients [4] replaces the L_2 distance with a cosine-similarity cost and adds a total-variation image prior:

$$\min_{\hat{x}} \left(1 - \frac{\langle g', g \rangle}{\|g'\| \, \|g\|}\right) + \alpha \, \mathrm{TV}(\hat{x}), \qquad g' = \nabla_w \, \ell(w_t; \hat{x}, y), \tag{4.3}$$

where $\mathrm{TV}(\hat{x}) = \sum_{i,j} |\hat{x}_{i+1,j} - \hat{x}_{i,j}| + |\hat{x}_{i,j+1} - \hat{x}_{i,j}|$ is the total variation of the image - the sum of absolute differences between horizontally and vertically neighboring pixels - which penalizes non-smooth reconstructions. The label y is recovered the same way as in iDLG (Equation (4.2)), and $\hat{x}$ is constrained to the valid image range $[0, 1]^d$ by a box constraint. Two choices make Equation (4.3) scale to deep networks where DLG's L_2 cost does not: the cosine-similarity term depends only on the *direction* of g', not its magnitude, which is otherwise dominated by whichever layer happens to have the largest gradient norm; and the optimizer applies Adam updates using only the *sign* of the computed gradient, which prevents any single coordinate's magnitude from controlling the step size. This project evaluates Equation (4.3) on an untrained ResNet-18 victim model, and separately on the LeNet architecture of Section 4.1, under the same weights and image used for DLG/iDLG; the latter configuration is the one used in the defense comparison of Section 4.5.

4.4 R-GAP

R-GAP [5] targets a feed-forward network of L bias-free layers with invertible elementwise activation σ: writing $a_0 = x$ for the input and, for $l = 1, \ldots, L$,

$$z_l = W_l \, a_{l-1}, \qquad a_l = \sigma(z_l), \tag{4.4}$$

the weight gradient at layer l for a single sample ($B = 1$) is the outer product

$$\frac{\partial \ell}{\partial W_l} = \frac{\partial \ell}{\partial z_l} \, a_{l-1}^{\top}. \tag{4.5}$$

The leaked gradient $\partial \ell / \partial W_l$ is known for every layer; $\partial \ell / \partial z_l$ is obtained recursively via the chain rule, $\partial \ell / \partial z_l = \sigma'(z_l) \odot \left(W_{l+1}^{\top} \, \partial \ell / \partial z_{l+1}\right)$, starting from the output layer, where $\partial \ell / \partial z_L$ is a scalar equation solved numerically (the loss-to-logit derivative is a monotonic function of the scalar logit for the binary logistic loss used here). With $\partial \ell / \partial z_l$ known, Equation (4.5) becomes a linear system for the unknown a_{l-1}, solved by least squares; recursing this from $l = L$ down to $l = 1$ recovers $a_0 = x$ - the private input itself - with *no* optimization loop: each layer's equation is solved exactly, subject to its own rank/invertibility condition. Convolutional layers are handled by first re-expressing the weight-sharing convolution as an equivalent dense (circulant) matrix multiplication, so that Equation (4.5) applies unchanged.

This project implements R-GAP on both victim architectures from Section 4.1 (LeNetRGAP and FCN3), and additionally runs the entire forward/backward pass and every linear solve in float64 rather than the default float32: because each layer's solution is an exact linear system rather than one term in an averaged loss, small floating-point rounding error does not cancel out as it would under gradient descent, and is instead compounded across the L recursive layers.

4.5 DP-SGD as a Defense

DP-SGD [6] is applied here, after the fact, as a gradient-perturbation defense: before the leaked gradient g reaches the attacker, it is clipped to a global L_2 norm C and perturbed with Gaussian noise of multiplier σ,

$$\tilde{g} = \text{clip}(g, C) + \mathcal{N}\big(0,\ (\sigma C)^2 I\big), \qquad \text{clip}(g, C) = g \cdot \min\left(1, \frac{C}{\|g\|_2}\right). \tag{4.6}$$

The same perturbed gradient $\tilde{g}$ is substituted wherever g appears in Equations (4.1), (4.3), and the R-GAP recursion of Section 4.4, allowing the three attacks to be compared under an identical defense budget (C, σ). Inverting Gradients is evaluated under both victim configurations introduced in Section 4.3: the ResNet-18 setup, and the LeNet architecture shared with DLG/iDLG. This project sweeps (C, σ) for each attack and configuration; the resulting reconstruction quality is reported in Chapter 5.

Chapter 5

Experimental Evaluation

5.1 Evaluation Objectives

5.2 Evaluation Scenarios

5.2.1 Datasets and Victim Configurations

5.2.2 Evaluation Metrics

5.3 Results and Discussion

5.3.1 RQ1 – Baseline Comparison on a Shared Architecture

5.3.2 RQ2 – Scaling to a Deeper Architecture

5.3.3 RQ3 – Robustness to Batch Size

5.3.4 RQ4 – Effectiveness of the DP-SGD Defense

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017), PMLR 54, pp. 1273–1282.

[2] L. Zhu, Z. Liu, S. Han. *Deep Leakage from Gradients*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019), pp. 14747–14756. arXiv:1906.08935.

[3] B. Zhao, K. R. Mopuri, H. Bilen. *iDLG: Improved Deep Leakage from Gradients*. arXiv:2001.02610, 2020.

[4] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller. *Inverting Gradients — How Easy Is It to Break Privacy in Federated Learning?* Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

[5] J. Zhu, M. B. Blaschko. *R-GAP: Recursive Gradient Attack on Privacy*. International Conference on Learning Representations (ICLR 2021). arXiv:2010.07733.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang. *Deep Learning with Differential Privacy*. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS 2016), pp. 308–318. arXiv:1607.00133.